

12.1 Functions and Their Computation

A **Function** is a mapping from inputs to outputs, where each input is assigned exactly one output.

To **compute a function** means to determine the output for a given input.

Ways to compute functions

- Table lookup (查表法)

Store all input-output pairs in a table and look them up.

Figure 12.1 An attempt to display the function that converts measurements in yards into meters

Yards (input)	Meters (output)
1	0.9144
2	1.8288
3	2.7432
4	3.6576
5	4.5720
.	.
.	.
.	.

-Algebraic formulas (公式法)

$$V=P(1+r)^n$$

-Advanced computation (高级计算)

$$\sin(38^\circ)$$

Even calculators do not compute sine directly—they approximate it using complex techniques.

Computable vs Noncomputable Functions

-Computable Function

A function is computable if there exists an algorithm that can compute its output for any input.

-Noncomputable Function

A function is noncomputable if no algorithm can compute it.

Recursive Function Theory (递归函数理论)

Recursive function theory studies problems that are beyond algorithmic solutions and analyzes their complexity.

1. Identify some functions that can be represented completely in tabular form.

Boolean functions (e.g., AND, OR, NOT)

Functions mapping days of the week to numbers (e.g., Monday $\rightarrow$ 1)

2. Identify some functions whose outputs can be described as an algebraic expression involving their inputs.

Addition: $f(x,y)=x+y$

Linear function: $f(x)=2x+3$

Compound interest: $V=P(1+r)^n$

3. Identify a function that cannot be described in terms of an algebraic formula. Is your function nonetheless computable?

An example is the sine function, such as $\sin(x)$, which cannot be described by a simple algebraic formula.

However, it is still computable, because its values can be approximated using algorithms (e.g., series expansions).

4. Ancient Greek mathematicians used a straight-edge and compass to draw shapes. They developed techniques for finding the midpoint on a straight line, constructing a right angle, and drawing an equilateral triangle. However, what were some “computations” that their “computational system” could not perform?

-三等分角 (Angle Trisection) -> 把任意角分成三等份

✗ 直尺+圆规做不到 (一般情况)

-倍立方 (Doubling the Cube) -> 已知一个立方体 → 构造一个体积是它 2 倍的立方体
需要构造 $\sqrt[3]{2}$

✗ 直尺+圆规无法构造这个数

-化圆为方 (Squaring the Circle) -> 画一个正方形, 使面积 = 给定圆
需要构造根号 π

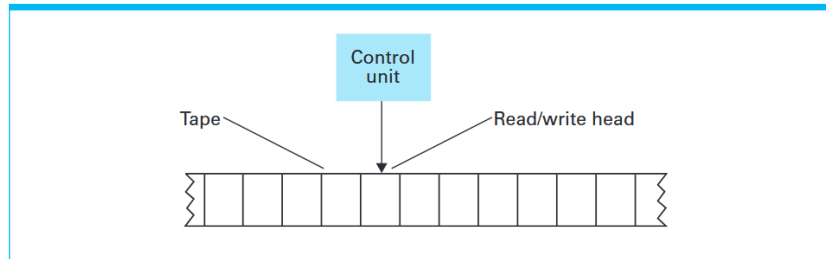
✗ π 是超越数 (transcendental) → 无法通过有限代数操作构造

12.2 Turing Machines

A **Turing machine** is an abstract computational model used to study the limits of algorithmic processes.

Turing Machine Fundamentals

Figure 12.2 The components of a Turing machine



An infinite **tape** divided into cells, each containing a symbol from a finite alphabet.

Read/Write Head

Control Unit

States: The machine is always in one of a finite number of states.

-START (开始) 你站在 * (数字最右边) 往左走一格, 准备开始

-ADD 对当前位 +1

--情况 1: 读到 0

$0 + 1 = 1$

结束 (不用进位)

--情况 2: 读到 1

$1 + 1 = 0$ (产生进位)

进入 CARRY

-CARRY

--遇到 0

$0 + 1 = 1$

进位结束 → RETURN

--遇到 1

$1 + 1 = 0$

继续往左进位 (还在 CARRY)

-RETURN

加完之后, 往右走 回到 *

-HALT (停止)

How computation works (计算如何进行)

Read symbol

Write symbol

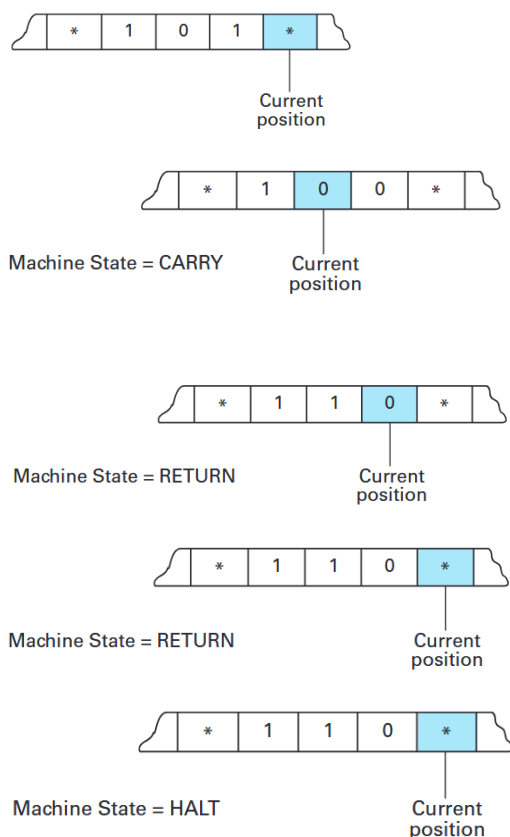
Move left/right

Change state

Example: Increment a Binary Number

Figure 12.3 A Turing machine for incrementing a value

Current state	Current cell content	Value to write	Direction to move	New state to enter
START	*	*	Left	ADD
ADD	0	1	Right	RETURN
ADD	1	0	Left	CARRY
ADD	*	*	Right	HALT
CARRY	0	1	Right	RETURN
CARRY	1	0	Left	CARRY
CARRY	*	1	Left	OVERFLOW
OVERFLOW	(Ignored)	*	Right	RETURN
RETURN	0	0	Right	RETURN
RETURN	1	1	Right	RETURN
RETURN	*	*	No move	HALT



The Church-Turing Thesis

Turing computable

A function is Turing computable if it can be computed by a Turing machine.

Example: Successor Function

The successor function maps $n \rightarrow n+1$.

101 * (=5)

110 * (=6)

用前面例子的图灵机可实现

Turing's Conjecture (图灵的猜想)

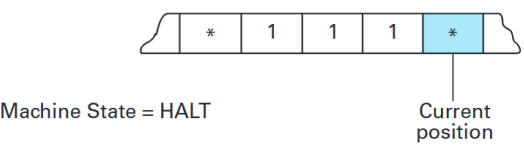
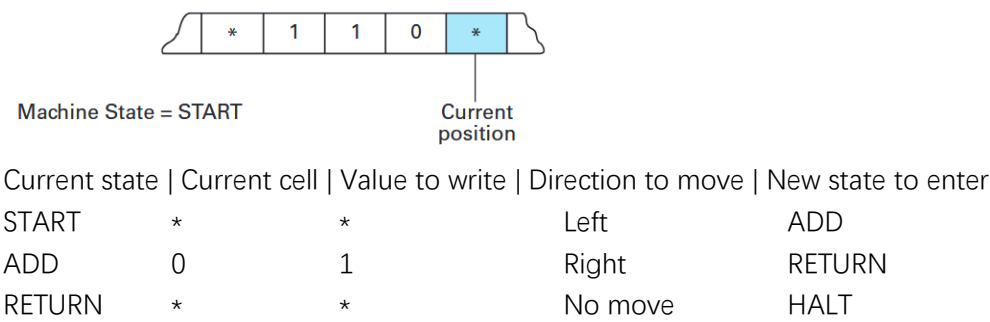
Turing-computable functions = all computable functions.

“凡是能用算法算的东西，都能用图灵机算” Algorithm = Turing Machine

Church-Turing thesis (丘奇-图灵论题)

The class of Turing-computable functions is exactly the class of computable functions.
所有“可计算函数” = 所有“图灵机能算的函数”

1. Apply the Turing machine described in this section (Figure 12.3), starting with the following initial status:



2. Describe a Turing machine that replaces a string of 0s and 1s with a single 0.

Current state	Current cell content	Value to write	Direction to move	New state to enter	
START	*	*	left	STATE 1	
STATE 1	0	0	left	STATE 2	
STATE 1	1	0	left	STATE 2	
STATE 1	*	0	left	STATE 2	
STATE 2	0	*	right	STATE 3	
STATE 2	1	*	right	STATE 3	
STATE 2	*	*	right	STATE 3	
STATE 3	0	0	right	HALT	
STATE 3	1	0	right	HALT	

Current state	Current cell content	Value to write	Direction to move	New state to enter	修改说明
START	*	*	right	STATE 1	往右找字符串开头
STATE 1	0	0	right	STATE 2	保留第一个数字为0，进清理状态
STATE 1	1	0	right	STATE 2	同上
STATE 2	0	*	right	STATE 2	关键：发现0就抹除，保持在STATE 2 循环
STATE 2	1	*	right	STATE 2	关键：发现1就抹除，保持在STATE 2 循环
STATE 2	*	*	-	HALT	遇到空白符，说明清理完毕，停止

感觉书上的这题写的不对，gemini 写的才对，把 right 全改成 left

3. Describe a Turing machine that decrements the value on the tape if it is greater than zero or leaves the value unaltered if it is zero.

Current state	Current cell content	Value to write	Direction to move	New state to enter
START	*	*	left	SUBTRACT
SUBTRACT	0	1	left	BORROW
SUBTRACT	1	0	left	NO BORROW
BORROW	0	1	left	BORROW
BORROW	1	0	left	NO BORROW
BORROW	*	*	right	ZERO
NO BORROW	0	0	left	NO BORROW
NO BORROW	1	1	left	NO BORROW
NO BORROW	*	*	right	RETURN
ZERO	0	0	right	ZERO
ZERO	1	0	right	ZERO
ZERO	*	*	no move	HALT
RETURN	0	0	right	RETURN
RETURN	1	1	right	RETURN
RETURN	*	*	no move	HALT

4. Identify an everyday situation in which calculating takes place. How is that situation analogous to a Turing machine?

An everyday situation is calculating the total cost while shopping. The person acts as the control unit, the written notes act as the tape, and reading and writing correspond to the operations of a Turing machine.

5. Describe a Turing machine that ultimately halts for some inputs but never halts for others.

Current state	Cell content	Value to write	Direction to move	New state to enter
START	*	*	left	STATE 1
STATE 1	0	0	right	HALT
STATE 1	1	1	no move	STATE 1
STATE 1	*	*	no move	STATE 1

最后一位是 0 → 偶数

最后一位是 1 → 奇数

12.3 Universal Programming Languages

A **universal programming language** is a language capable of expressing all Turing-computable functions.

能计算所有“图灵可计算函数”的语言

The Bare Bones Language

(1) clear

clear X sets X to 0.

(2) incr

incr X increases X by 1.

(3) decr

decr X decreases X by 1, but never below 0.

Control Structure: while (唯一控制结构)

while X not 0:

...

无限循环例子

incr X

while X not 0:

incr Z

正确例子

clear Z

while X not 0:

incr Z

decr X

$X \rightarrow 0$

$Z \rightarrow$ 原来 X 的值

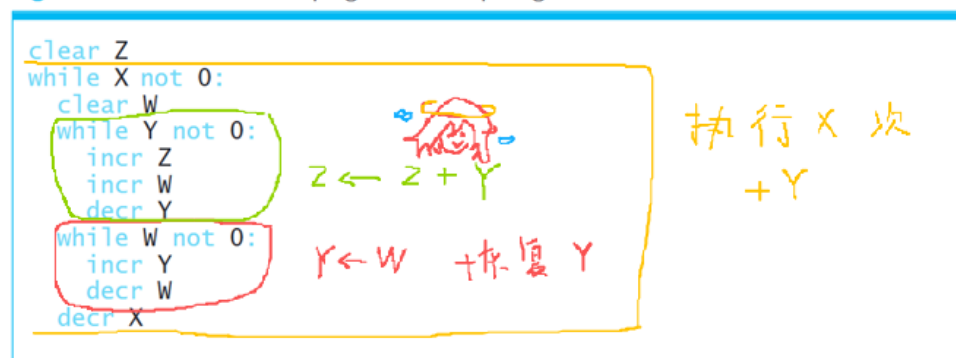
实现复制 (copy)

Important Example: Multiplication

Bare Bones can compute the product of X and Y.

$Z = X \times Y$

Figure 12.4 A Bare Bones program for computing $X \times Y$



Programming in Bare Bones

First Example: Assigning a constant value

clear X

incr X

incr X

incr X

Copy Problem

clear Aux

clear Z

while X not 0:

 incr Aux 让 Aux 做 X 的替身

 decr X

while Aux not 0:

 incr X 把 X 给恢复了

 incr Z

 decr Aux

Temporary storage

Aux acts as temporary storage to preserve data during computation.

Figure 12.5 A Bare Bones implementation of the instruction “copy Today to Tomorrow”

```
clear Aux
clear Tomorrow
while Today not 0:
    incr Aux
    decr Today
while Aux not 0:
    incr Today
    incr Tomorrow
    decr Aux
```

Handwritten notes:

- $Aux \leftarrow Today$ (red)
- Today 恢复 (green)
- $Tomorrow \leftarrow Aux$ (green)

The Universality of Bare Bones

Example

incr X

$f(X) = X + 1$

Second Example: Addition (加法)

copy Y to Z

while X not 0:

 incr Z

 decr X

实现了 $Z = X + Y$

Bare Bones programs = functions

Bare Bones can simulate Turing-computable functions

Church-Turing thesis: Turing-computable = all computable functions

1. Show that the statement invert X (whose action is to convert the value of X to zero if its initial value is nonzero and to 1 if its initial value is zero) can be simulated by a Bare Bones program segment.


```

clear AUX
incr AUX 1
while X not 0:
  clear X 0
  clear AUX 0
  while AUX not 0:
    incr X 1
    clear AUX

```

Handwritten notes: X=0, X=0

2. Show that even our simple Bare Bones language contains more statements than necessary by showing that the clear statement can be replaced with combinations of other statements in the language.

```

while X not 0:
  decr X

```

3. Show that the if-else structure can be simulated using Bare Bones. That is, write a program sequence in Bare Bones that simulates the action of the statement

if X not 0:

S1

else:

S2

where S1 and S2 represent arbitrary statement sequences.

```

copy X to AUX      AUX ← X
while AUX not 0:
  S1
  clear AUX
copy X to AUX      AUX ← X
invert AUX (See Question #1)
while AUX not 0:
  S2
  clear AUX
while X not 0:
  clear AUX
  clear X

```

Handwritten notes: If AUX = 0 → 1, 1 → 0

4. Show that each of the Bare Bones statements can be expressed in terms of the machine language of Appendix C. (Thus Bare Bones can be used as a programming language for such a machine.)

If we assume that X refers to the memory cell at address 40 and that each program segment starts at location 00, we have the following conversion table:

```

clear X
incr X
decr X
while X not 0:

```

.

.

.

5. How can negative numbers be dealt with in Bare Bones?

Just as in a real machine, negative numbers could be dealt with via a coding system. For example, the rightmost bit in each string can be used as a sign but with the remaining bits used to represent the magnitude of the value.

6. Describe the function computed by the following Bare Bones program, assuming the function's input is represented by X and its output by Z:

```
clear Z
while X not 0:
    incr Z
    incr Z
    decr X
Z=2X
```

12.4 A Noncomputable Function

The Halting Problem

halting problem asks whether a program will eventually stop (halt) or run forever for a given input.

while X not 0:

 incr X

Case 1: X = 0

while 不执行 → 程序结束

Case 2: X ≠ 0

每轮:

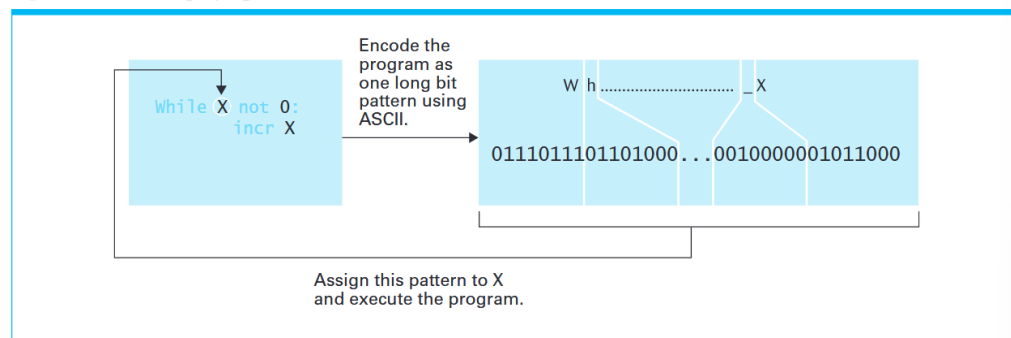
X 增加

永远不可能变成 0

无限循环

self-reference means a program refers to itself.

Figure 12.6 Testing a program for self-termination



X ≠ 0 (因为 N 很大)

while 条件成立

每次 incr X → X 更大

永远不会变成 0

无限循环

clear X

while X not 0:

 incr X

X = encode(这个程序)

执行:

1. clear X → X = 0
2. while 不执行

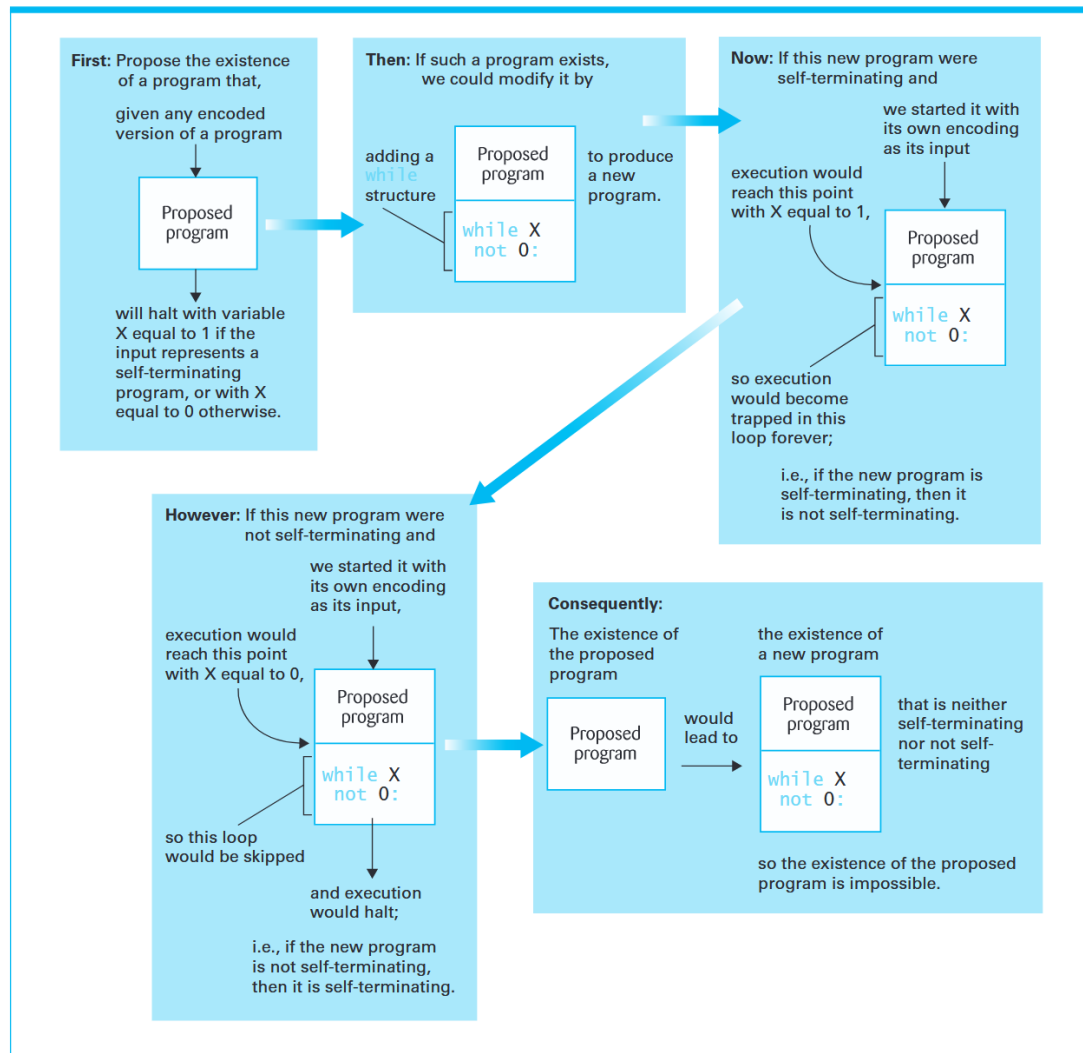
程序结束

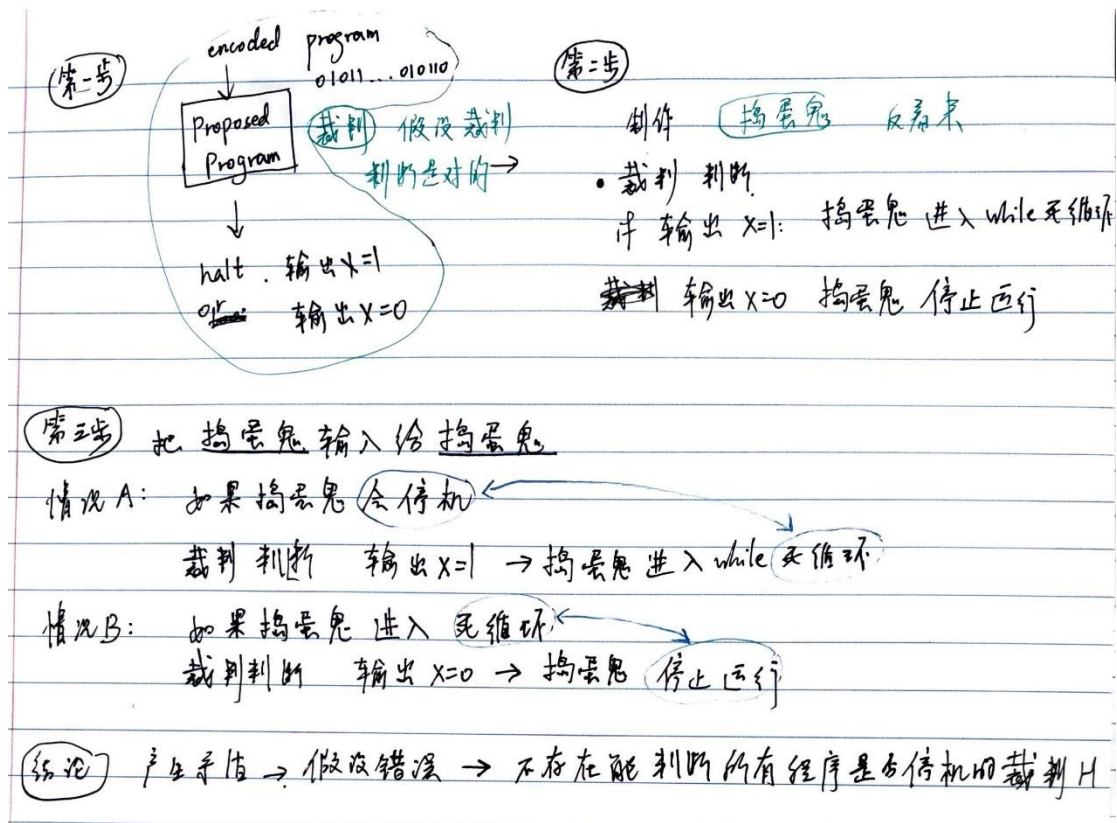
self-terminating if it halts when given its own encoding as input.

The halting problem can now be defined as determining whether a program is self-terminating.

The Unsolvability of the Halting Problem

Figure 12.7 Proving the unsolvability of the halting program





unsolvable problems are problems for which no algorithm exists.

1. Is the following Bare Bones program self-terminating? Explain your answer.

```
incr X
decr Y
```

Yes. In fact, this program halts regardless of the initial values of its variables, and therefore it must halt if its variables are initialized to the program's encoded representation.

2. Is the following Bare Bones program self-terminating? Explain your answer.

```
copy X to Y
incr Y
incr Y
while X not 0:
    decr X
    decr X
    decr Y
    decr Y
decr Y
while Y not 0:
```

如果 X=1, 会停机, 遇到奇数会停机。

当它读自己时, 它读入的序列末尾就是分号 (因为 bare bone 的语句以结尾)。分号的二进制末尾是 1 程序逻辑规定末尾是 1 就停。结果: 自停机。

3. What is wrong with the following scenario?

In a certain community, everyone owns his or her own house. The house painter of

the community claims to paint all those and only those houses that are not painted by their owners.

(Hint: Who paints the house painter's house?)

The point here is that the logic is the same as in our argument that the halting problem does not have an algorithmic solution. If the house painter paints his or her own house, then he or she does not and vice versa.

12.5 Complexity of Problems

Measuring a Problem's Complexity

The **time complexity** of a problem is defined as $\Theta(f(n))$ if there exists an algorithm with time complexity $\Theta(f(n))$ and no algorithm solves the problem faster.

big O notation

If a problem can be solved by an algorithm in $\Theta(f(n))$, then the problem is in $O(f(n))$.

Θ = best possible

O = we only know an upper bound

4. 一个具体的例子

假设有一个程序，它的实际执行步骤是 $10n + 5$:

1. 它可以是 $O(n)$ 吗？**可以**。(它是线性增长的，没有超过 n 的阶数)
2. 它可以是 $O(n^2)$ 吗？**可以**。(虽然它比 n^2 快得多，但 n^2 确实是它的一个有效“上界”)
3. 它可以是 $\Theta(n)$ 吗？**可以**。(它精确地属于线性阶)
4. 它可以是 $\Theta(n^2)$ 吗？**不可以!** (因为它的增长速度没有达到 n^2 的量级， Θ 要求必须“门当户对”)

Example 1: Searching

Binary Search: $T(n) = \Theta(\log_2 n)$ 所以 The searching problem is in $O(\log n)$.

实际上: 搜索问题 $\in \Theta(\log n)$

binary search 是最优解

Example 2: Sorting

Insertion Sort: $T(n) = \Theta(n^2)$ 所以排序问题 $\in O(n^2)$

但实际上: 排序问题 $\in \Theta(n \log n)$

insertion sort 不是最优解

Merge Sort

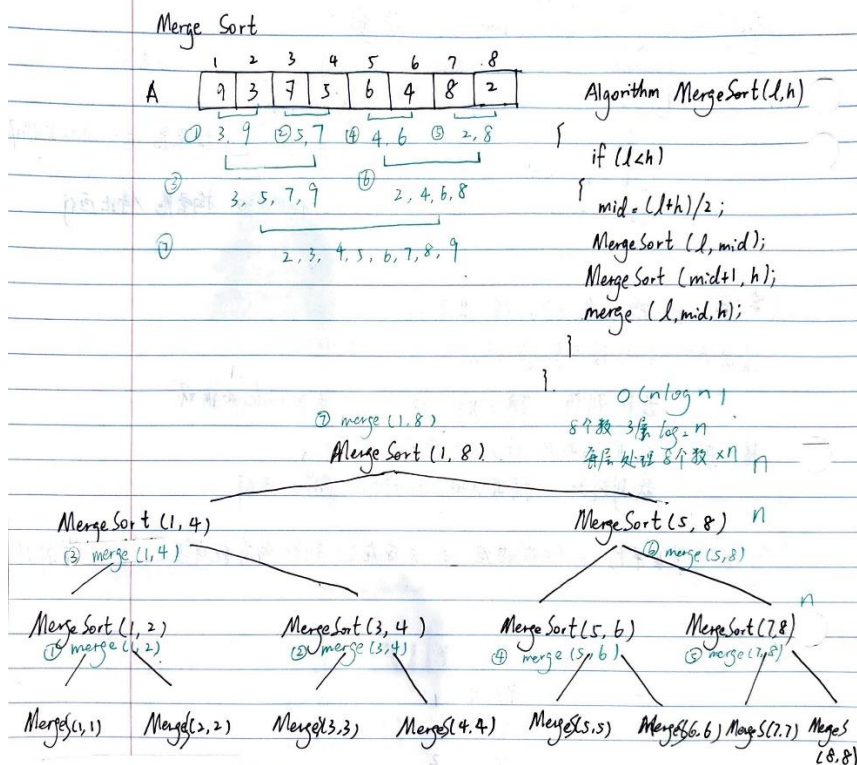
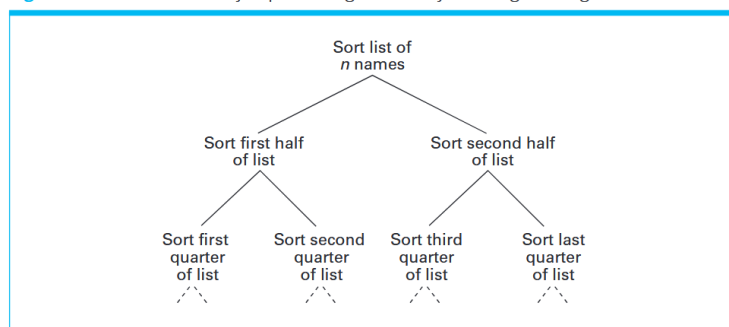


Figure 12.10 The hierarchy of problems generated by the merge sort algorithm



Space Complexity

Space complexity measures the amount of memory required to solve a problem.

Example: Sorting Problem $O(n)$

Insertion Sort $O(n)$

空间复杂度 $\leq$ 时间复杂度

Polynomial versus Nonpolynomial Problems

polynomial problem (多项式问题)

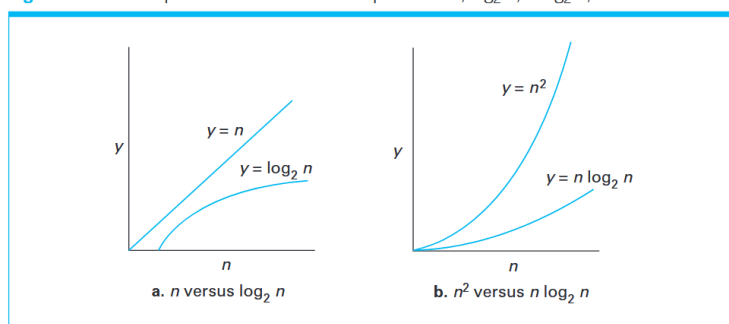
A problem is a polynomial problem if it is in $O(f(n))$, where $f(n)$ is a polynomial or bounded by a polynomial. 复杂度 $\leq$ 多项式 (如 $n, n^2, n^3 \dots$)

Class P

The set of all polynomial problems is called P. 所有“多项式时间可解”的问题

Problems in P are considered practically solvable.

Figure 12.11 Graphs of the mathematical expressions n , $\log_2 n$, $n \log_2 n$, and n^2



NP Problems (Nonpolynomial Problems (非多项式问题))

Exponential functions are not bounded by any polynomial. 2^n

A **deterministic algorithm** specifies exactly what to do at each step.

A **nondeterministic algorithm** can “choose” among multiple possibilities without specifying how.

Traveling Salesperson Problem

从家出发：访问所有城市、回到起点、总距离 $\leq$ 限制。

暴力解法 (deterministic) 枚举所有路径：比较每一条路径

超指数增长 (接近 $n!$) 非多项式时间 $\rightarrow$ impractical

nondeterministic algorithm

“直接猜一个路径” 如果刚好猜对：很快结束

这个过程：选路径： $O(n)$ 计算距离： $O(n)$

总复杂度： $O(n)$ 是多项式时间 $\checkmark$ (容易验证答案的意思)

A problem that can be solved in polynomial time by a nondeterministic algorithm is called a

nondeterministic polynomial problem, or an **NP problem** for short. It is customary to denote the class of NP problems by **NP**.

$P \subseteq NP$

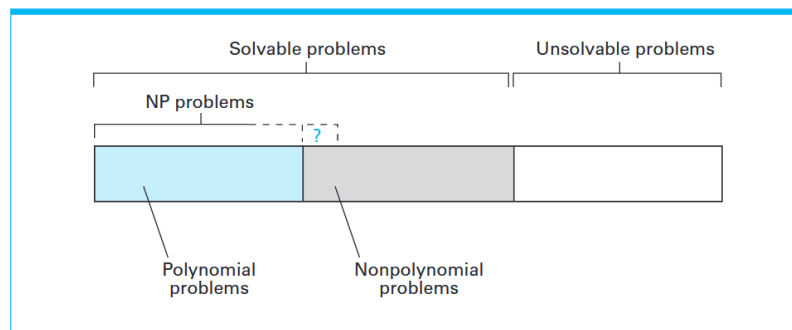
NP-complete problems are the hardest problems in NP.

If one NP-complete problem can be solved in polynomial time, then all NP problems can.

Traveling Salesperson Problem $\in$ NP-complete

Efficient solutions to NP problems would break many encryption systems.

Figure 12.12 A graphic summation of the problem classification



1. Suppose a problem can be solved by an algorithm in $\Theta(2^n)$. What can we conclude about the complexity of the problem?

If a problem can be solved by an algorithm in $\Theta(2^n)$, then the problem is in $O(2^n)$, but we cannot conclude that its complexity is $\Theta(2^n)$ because a more efficient algorithm may exist.

2. Suppose a problem can be solved by an algorithm in $\Theta(n^2)$ as well as another algorithm in $\Theta(2^n)$. Will one algorithm always outperform the other?

No, one algorithm will not always outperform the other. Although the $\Theta(n^2)$ algorithm is more efficient for large values of n , the $\Theta(2^n)$ algorithm may perform better for small values of n due to constant factors.

假设：

- 算法 A: $1000 \cdot n^2$
- 算法 B: 2^n

👉 当 $n = 5$:

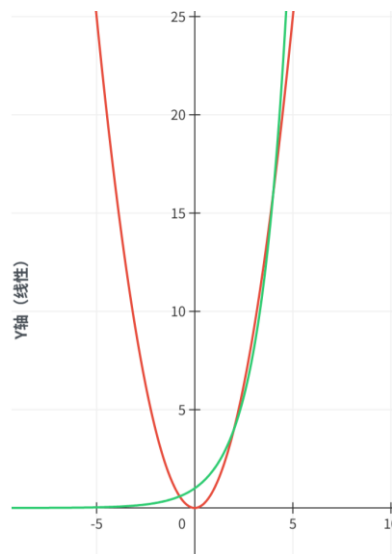
- $A = 1000 \times 25 = 25000$
- $B = 32$

👉 B 更快 ✓

👉 当 $n = 20$:

- $A = 1000 \times 400 = 400000$
- $B \approx 1,000,000$

👉 A 更快 ✓



3. List all of the subcommittees that can be formed from a committee consisting of the two members Alice and Bill. List all the subcommittees that can be formed from the committee consisting of Alice, Bill, and Carol. What about the subcommittees from Alice, Bill, Carol, and David?

Alice, Bill

{Alice}

{Bill}

{Alice, Bill}

$$2^2 - 1 = 3$$

Alice, Bill, Carol

{Alice}

{Bill}

{Carol}

{Alice, Bill}

{Alice, Carol}

{Bill, Carol}

{Alice, Bill, Carol}

$$2^3 - 1 = 7$$

Alice, Bill, Carol, David

单人：

{Alice}

{Bill}

{Carol}

{David}

两人：

{Alice, Bill}

{Alice, Carol}

{Alice, David}

{Bill, Carol}

{Bill, David}

{Carol, David}

三人:

{Alice, Bill, Carol}

{Alice, Bill, David}

{Alice, Carol, David}

{Bill, Carol, David}

四人:

{Alice, Bill, Carol, David}

$$2^4 - 1 = 15$$

4. Give an example of a polynomial problem. Give an example of a nonpolynomial problem. Give an example of an NP problem that as yet has not been shown to be a polynomial problem.

P binary search $O(\log_2 n)$

NP The problem of listing all subcommittees of a group of n people. $2^n - 1$

NP Problem (not known to be polynomial) TSP

5. If the complexity of algorithm X is greater than that of algorithm Y, is algorithm X necessarily harder to understand than algorithm Y? Explain your answer.

No. Time complexity measures the number of steps required by an algorithm, not its conceptual difficulty. An algorithm with higher time complexity may still be easier to understand than one with lower complexity.

12.6 Public-Key Cryptography

大整数分解 (factoring large integers) 很难, 利用这点来保护信息。

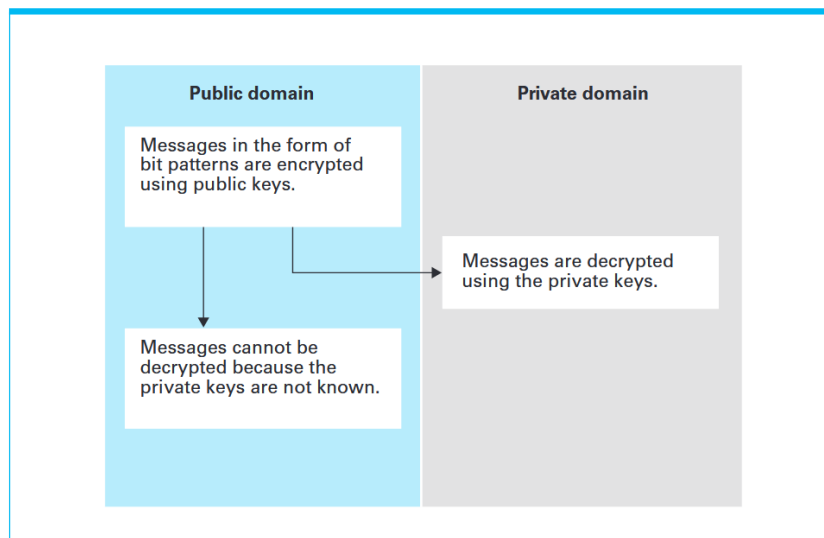
RSA algorithm is a public-key encryption method based on the difficulty of factoring large integers.

encrypting keys → used to encrypt messages **public keys**

decrypting keys → used to decrypt messages **private keys**

A public-key encryption system allows the encrypting keys to be public while keeping the decrypting keys private.

Figure 12.13 Public key cryptography



Even though the public key is known, deriving the private key requires solving a difficult problem (factoring large integers).

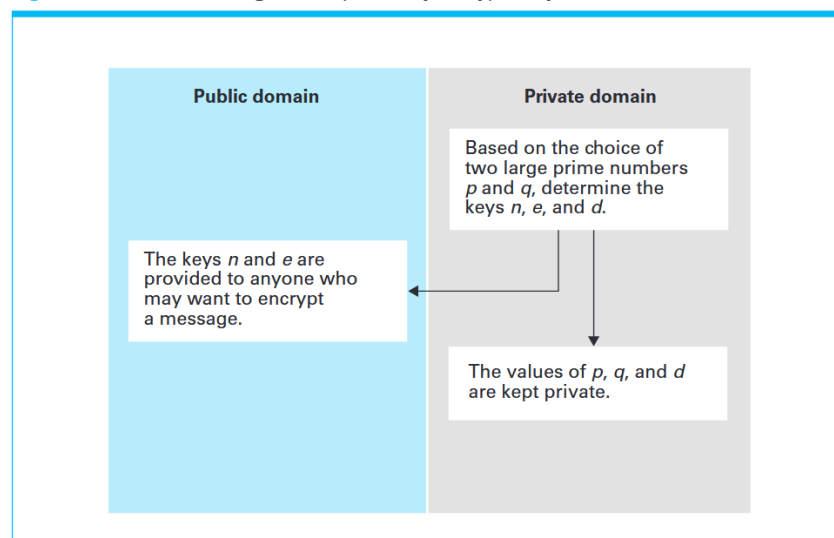
Modular Notation

The notation $x \% m$ (or $x \bmod m$) represents the remainder when x is divided by m .

$$m^{k(p-1)(q-1)} \equiv 1 \pmod{pq}$$

RSA Public-Key Cryptography

Figure 12.14 Establishing an RSA public key encryption system



RSA		How does it work?	
Encryption	$(5, 14)$	① $p=2, q=7$	①
Text	$B \rightarrow 2$	② $N=p \times q = 2 \times 7 = 14$	②
	$2^5 \pmod{14} = 32 \pmod{14}$	③ $\phi(N) = (p-1)(q-1)$	③
	$= 4 \pmod{14}$	$= 1 \times 6$	④
	Cypher text: $D \rightarrow 4$	$= 6$	⑤
Decryption	$(11, 14)$	④ choose e $\begin{cases} 1 < e < \phi(N) \\ \text{coprime with } N, \phi(N) \end{cases}$	⑥
	$D \rightarrow 4^5 \pmod{14} = 1024 \pmod{14}$	$e=5 \rightarrow \text{Lock}(5, 14)$	⑦
	$= 2 \pmod{14}$	⑤ choose d : $de \pmod{\phi(N)} = 1$	⑧
	$\downarrow B$	$5d \pmod{6} = 1$	⑨
		$5, 10, 15, 20, 25, 30, \dots$	⑩
		$\pmod{6} \quad 5, 4, 3, 2, 1, 0, 1, 2, \dots$	⑪
		$\uparrow$	⑫
		KEY (11, 14)	⑬
		the number coprime with 14	
		和 14 互素	

- Find the factors of 66,043. (Don't waste too much time on this one. The point is that it can be time-consuming.)

$$66,043 = 211 \times 313$$

我体会到了，分解因数确实太费时间了，所以 RSA 很安全。

- Using the public keys $n = 91$ and $e = 5$, encrypt the message 101.

$$\begin{aligned}
 &2. \quad n=91 \quad e=5 \\
 &\quad \text{Lock}(5, 91) \\
 &101 \rightarrow 5^5 \pmod{91} = 64 \\
 &\quad \downarrow \\
 &\quad 1000000
 \end{aligned}$$

- Using the private keys $n = 91$ and $d = 29$, decrypt the message 10.

$$\begin{aligned}
 &3. \quad n=91 \quad d=29 \\
 &\quad \text{KEY}(29, 91) \\
 &10 \rightarrow 2^{29} \pmod{91} = 32 \rightarrow 1000000
 \end{aligned}$$

4. Find the appropriate value for the decrypting keys n and d in an RSA public-key cryptography system based on the primes $p = 7$ and $q = 19$ and the encryption key $e = 5$.

$$\begin{aligned} 4. \quad & p=7 \quad q=19 \quad e=5 \\ & N = 7 \times 19 = 133 \quad \phi(N) = 6 \times 18 = 108 \\ & \text{choose } d: \quad de \pmod{\phi(N)} = 1 \\ & \quad \quad \quad 5d \pmod{108} = 1 \\ & \quad \quad \quad d = \frac{108m + 1}{5} \\ & m=3, \quad d=65 \quad (65, 133) \end{aligned}$$